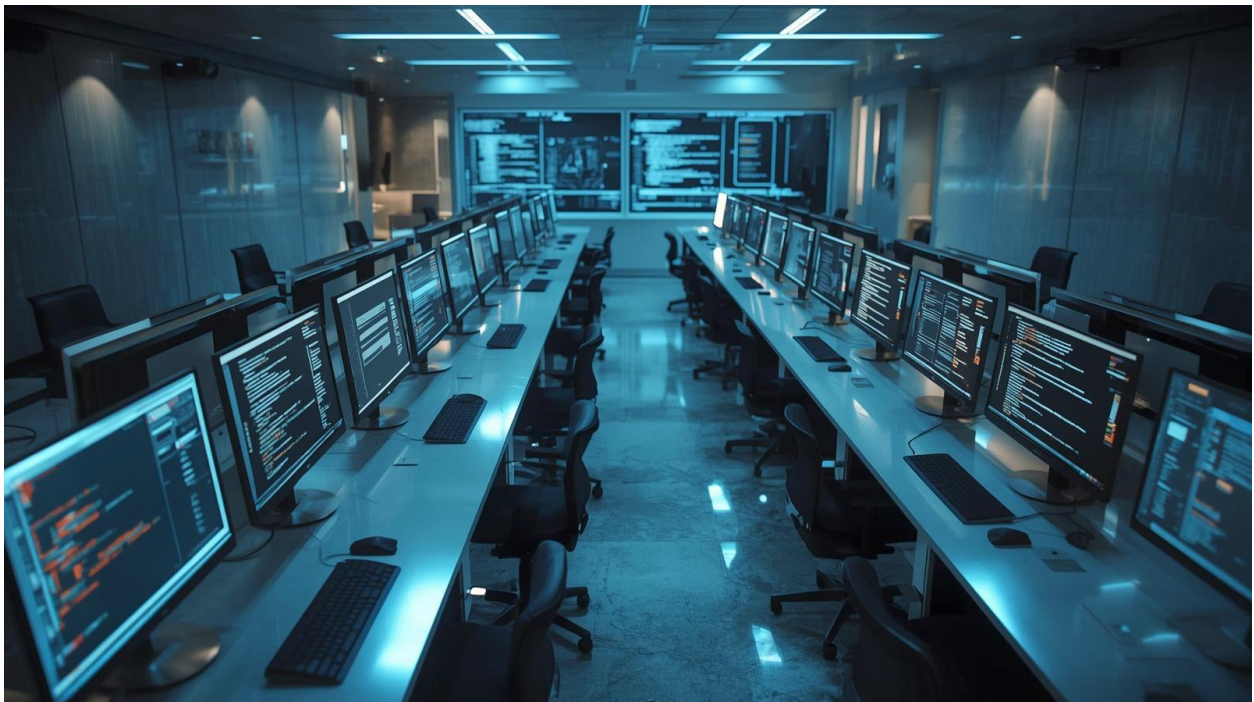


SAE 501 : Concevoir, réaliser et présenter une solution technique

RAPPORT DE SAE



Sailliot Valentin, Duforets Thomas, Guyot Loïc

Sommaire

Projet de salles de TP virtuelles à l'aide de machines virtuelles	2
Sujet de Projet: Configuration et Déploiement d'une Infrastructure de Salles de TP Virtuelles.....	2
Objectifs:	2
Plan du projet:	2
Phase 1: Conception et Planification (10%)	2
Phase 2: Installation et Configuration de l'Hyperviseur (25%)	2
Phase 3: Création des Machines Virtuelles Modèles (20%)	2
Phase 4: Interface Web pour l'Accès à Distance (25%)	2
Phase 5: Documentation et Formation (10%).....	2
Phase 6: Tests et Validation (5%)	2
Phase 7: Présentation et Livraison (5%)	2
Barème:.....	3
Ressources transversales:.....	3
Évaluation:.....	3
Outils Suggérés:	3
Ressources :	3
Choix de l'hyperviseur pour le projet « Salles de TP virtuelles »	5
Synthèse	8
Répartition des tâches et des responsabilités.....	8
Établissement du calendrier et des jalons	9
Installation de l'hyperviseur	11
Configuration des ressources	11
1. Configuration du Processeur (CPU).....	11
2. Configuration de la Mémoire Vive (RAM)	12
3. Configuration Réseau (Network).....	12
Définition des besoins en logiciels et environnement pour les TP.....	14
Création et configuration des machines virtuelles modèles	15
Déployer une solution utilisateur pour accéder aux VM de n'importe où.....	17
Mise en oeuvre de l'authentification et de la gestion des droits	18
Tests de performance et de sécurité	21
Validation fonctionnelles de l'infrastructure.....	21
Réflexion sur les aspects de développement durables et de transition énergétique	27
Synthèse en anglais	28
Executive Summary: Virtual Practical Work Rooms Infrastructure	28

Projet de salles de TP virtuelles à l'aide de machines virtuelles

Conditions d'achèvement

Sujet de Projet: Configuration et Déploiement d'une Infrastructure de Salles de TP Virtuelles

Objectifs:

1. Installer et configurer un hyperviseur sur un serveur dédié.
2. Concevoir des machines virtuelles modèles pour les TP avec les logiciels et environnements nécessaires.
3. Déployer une interface web pour l'accès à distance aux machines virtuelles.
4. Intégrer les aspects de sécurité (authentification, gestion d'accès, etc.).
5. Mettre en place une documentation et des procédures pour la maintenance et la supervision.

Plan du projet:

Phase 1: Conception et Planification (10%)

- Choix de l'hyperviseur (ex: VMware, Hyper-V, KVM, Proxmox)
- Répartition des tâches et des responsabilités (3-4 étudiants par groupe)
- Établissement du calendrier et des jalons

Phase 2: Installation et Configuration de l'Hyperviseur (25%)

- Installation de l'hyperviseur sur un serveur
- Configuration des ressources (CPU, mémoire, réseau, utilisateurs, ...)

Phase 3: Création des Machines Virtuelles Modèles (20%)

- Définition des besoins en logiciels et environnements pour les TP
- Création et configuration des machines virtuelles modèles

Phase 4: Interface Web pour l'Accès à Distance (25%)

- Déployer une solution utilisateur (guacamole, bureau à distance ...). Les étudiants doivent pouvoir accéder aux VM via l'interface web de n'importe où.
- Mise en œuvre de l'authentification et de la gestion des droits

Phase 5: Documentation et Formation (10%)

- Rédaction des manuels et procédures

Phase 6: Tests et Validation (5%)

- Tests de performance et de sécurité
- Validation fonctionnelle de l'infrastructure

Phase 7: Présentation et Livraison (5%)

- Présentation à l'équipe pédagogique
- Livraison et déploiement

Barème:

- Phase 1: 10 points
- Phase 2: 25 points
- Phase 3: 20 points
- Phase 4: 25 points
- Phase 5: 10 points
- Phase 6: 5 points
- Phase 7: 5 points

Ressources transversales:

- Synthèse en anglais: 5 points bonus
- Réflexion sur les aspects de développement durable et de transition énergétique: 5 points bonus

Évaluation:

- 3-4 étudiants par groupe.
- Le projet sera évalué par des enseignants
- Évaluations individuelles et de groupe pour chaque phase.

Outils Suggérés:

- Outils de gestion de projet: Gitlab, Trello, Oktonine
- Systèmes de gestion de versions: Git
- Outils de virtualisation: Proxmox, VMware, Hyper-V, KVM

Ressources :

- <https://tel.marionnet.org/info.php>
- <https://www.marionnet.org/site/index.php/fr/>
- https://pve.proxmox.com/wiki/Proxmox_VE_inside_VirtualBox
- <https://guacamole.apache.org>

Phase 1 : Conception et Planification (10%)

Choix de l'hyperviseur pour le projet « Salles de TP virtuelles »

1. VMware vSphere / ESXi

- **Avantages**
 - Très robuste et reconnu en entreprise.
 - Fonctionnalités avancées (vMotion, HA, DRS).
- **Inconvénients pour ce projet**
 - **Coût élevé** → licences payantes, peu adaptées à un projet pédagogique étudiant.
 - Complexité inutile pour une salle de TP (surcouche lourde).
 - Rachat par Broadcom : incertitudes et prix en forte hausse.
- **Conclusion** : adapté aux grandes infrastructures professionnelles, mais disproportionné pour un projet académique limité en budget.

2. Microsoft Hyper-V

- **Avantages**
 - Bien intégré dans un environnement 100% Windows.
 - Gratuit avec Windows Server.
- **Inconvénients pour ce projet**
 - Peu flexible avec Linux et environnements hétérogènes.
 - Interface de gestion moins complète que Proxmox.
 - Pas d'outils intégrés pour la gestion centralisée de clusters ou de conteneurs.
- **Conclusion** : bon choix si le TP devait être orienté Windows Server, mais peu adapté pour une salle multi-OS et ouverte.

3. KVM (pur)

- **Avantages**
 - Hyperviseur open source très puissant et performant.
 - Utilisé par de grands projets (OpenStack, Red Hat).
- **Inconvénients pour ce projet**
 - Pas d'interface graphique complète par défaut.
 - Administration complexe (ligne de commande, `virsh`, `libvirt`).
 - Pas de gestion native de clusters, backups ou accès web → il faut tout construire autour.

- **Conclusion** : techniquement solide, mais trop lourd en gestion pour des étudiants qui doivent livrer un projet pédagogique clé en main.

4. Proxmox VE

- **Avantages**
 - Gratuit et open source.
 - Interface web très simple et complète.
 - Basé sur KVM pour les VM et LXC pour les conteneurs.
 - Gestion intégrée des snapshots, sauvegardes, utilisateurs et droits d'accès.
 - Support du clustering et de la haute disponibilité (si plusieurs serveurs).
 - Large communauté et documentation abondante.
- **Inconvénients**
 - Moins reconnu officiellement que VMware en entreprise.
 - Certaines fonctionnalités avancées demandent un peu de configuration Linux.
- **Conclusion** : le choix idéal pour ce projet : gratuit, pédagogique, accessible via une interface web et avec toutes les fonctionnalités nécessaires (multi-OS, gestion des droits, snapshots, facilité d'accès).

Critères	VMware vSphere / ESXi	Microsoft Hyper-V	KVM (pur)	Proxmox VE
Coût	Très cher (licences payantes, surtout pour HA et gestion avancée)	Gratuit avec Windows Server, mais fonctionnalités avancées → payantes	Gratuit (open source)	Gratuit (open source, licence entreprise optionnelle)
Complexité d'administration	Complexe, nécessite des compétences avancées	Assez simple pour Windows, limité hors Microsoft	Administration en ligne de commande, pas d'interface	Interface web intégrée, facile à gérer

			complète par défaut	
Fonctionnalités intégrées	Très riche (HA, vMotion, DRS, etc.)	Fonctionnalités de base, extensions via System Center (payant)	Minimaliste sans surcouche	Snapshots, sauvegardes, clustering, gestion utilisateurs inclus
Compatibilité OS invités	Large (Windows, Linux, BSD)	Optimisé pour Windows, Linux supporté mais moins fluide	Large (Linux, Windows, BSD)	Large (Linux, Windows, BSD)
Interface d'accès	vSphere Client (riche, mais licence)	Hyper-V Manager (basique)	dVirt-Manager ou CLI (limité)	Interface web intuitive
Accès web pour utilisateurs (TP)	Possible via solutions tierces (payantes ou complexes)	Nécessite RDP, pas d'accès web natif	Doit être ajouté manuellement	Compatible avec Guacamole et accès web intégré
Support & communauté	Support officiel VMware (payant)	Support Microsoft, communauté modérée	Communauté Linux solide	Très grande communauté active
Pertinence pour le projet	Trop coûteux et lourd pour un contexte pédagogique	OK seulement si l'environnement est 100% Windows	Trop complexe à administrer pour des étudiants	Meilleur compromis (simple, complet, gratuit)

Synthèse

Pour un projet académique comme « **Salles de TP virtuelles** », l'hyperviseur doit être :

- **simple à administrer** (les étudiants doivent pouvoir gérer sans formation trop lourde),
- **ouvert et flexible** (support Linux et Windows),
- **peu coûteux** (licence gratuite ou open source),
- **compatible avec un accès web** (pour les étudiants à distance).

Proxmox VE est le meilleur choix : il combine la puissance de KVM avec une interface conviviale, intègre directement la gestion de snapshots, backups et droits, et se connecte facilement pour l'accès à distance.

Répartition des tâches et des responsabilités

Matrice RACI :

Matrice RACI	Thomas Duforets	Valentin Sailliot	Loïc Guyot
Répartition des tâches	R	R	R
Établissement du calendrier et des jalons	C	C	R
Installation de l'hyperviseur sur un serveur	R	C	C
Configuration des ressources	R	C	C
Définition des besoins en logiciels et environnements pour les TP	C	R	R
Création et configuration des machines virtuelles modèles	A	R	C
Déployer une solution utilisateur	R	A	A
Mise en œuvre de l'authentification et de la gestion des droits	A	A	R
Tests de performance et de sécurité	R	A	A
Validation fonctionnelle de l'infrastructure	R	R	R

Afin de structurer notre collaboration et d'assurer le bon déroulement du projet, nous avons défini les rôles de chacun à travers une matrice RACI. Cette répartition met en évidence une gestion de projet collaborative où la répartition des tâches et la validation fonctionnelle sont partagées par l'ensemble de l'équipe.

Établissement du calendrier et des jalons

Diagramme GANTT :

Tr	Tâche	État	Date de début	Date de fin
	Phase 1 (Conception et Planification)	Terminé	17/09/2025	17/09/2025
	Phase 2 (Installation et Configuration de l'Hyperviseur)	Terminé	19/09/2025	29/09/2025
	Phase 3 (Création des Machines Virtuelles Modèles)	Terminé	29/09/2025	12/11/2025
	Phase 4 (Interface Web pour l'Accès à Distance)	Terminé	17/10/2025	26/11/2025
	Phase 5 (Documentation et Formation)	Terminé	17/10/2025	05/12/2025
	Phase 6 (Tests et Validation)	Terminé	02/12/2025	05/12/2025
	Phase 7 (Présentation et Livraison)	En cours	09/12/2025	09/12/2025

Le projet a été rigoureusement planifié et structuré en **sept phases clés**, s'étalant du **17 septembre au 9 décembre 2025**. Cette approche séquencée a permis d'assurer un suivi précis de l'avancement et le respect strict des délais établis.

Comme l'illustre le diagramme de Gantt ci-dessous, la progression a suivi une logique en trois étapes majeures :

- **Phase de Socle Technique (Septembre - Novembre)** : Après la courte phase de conception initiale, l'équipe s'est focalisée sur l'installation et la configuration de l'hyperviseur, suivie par la longue phase de création des machines virtuelles modèles. Ces étapes critiques d'infrastructure ont été finalisées dans les temps, posant la base stable de la solution.
- **Parallélisation et Finitions (Octobre - Décembre)** : Afin d'optimiser le temps, le développement de l'interface Web et la rédaction de la documentation/formation ont été menés en parallèle dès le 17 octobre. Cette gestion simultanée des tâches a permis de gagner en efficacité et de respecter la date butoir.
- **Clôture et Livraison (Décembre)** : Le projet s'est achevé par une phase dédiée aux tests et à la validation (du 2 au 5 décembre), garantissant la qualité du livrable. Actuellement, seule la phase finale de **Présentation et Livraison** est en cours, marquant l'aboutissement réussi de la planification initiale.

Phase 2 : Installation et Configuration de l'Hyperviseur (25%)

Installation de l'hyperviseur

Afin de déployer notre infrastructure dans l'environnement disponible, nous avons opté pour une architecture de **virtualisation imbriquée**. L'hyperviseur **Proxmox VE** a ainsi été installé en tant que machine virtuelle hébergée sur **VMware**.

Cette méthode nous a permis de simuler un serveur physique complet tout en conservant la flexibilité de l'environnement VMware. Une attention particulière a été portée à la configuration de la machine virtuelle hôte pour permettre à Proxmox d'exécuter lui-même des machines virtuelles de manière performante.

La procédure technique, détaillant les paramètres spécifiques appliqués dans VMware et l'installation de l'ISO Proxmox, est disponible en **Annexe**. Une fois l'installation finalisée, la connectivité du "serveur" Proxmox a été validée via son interface web d'administration.

Configuration des ressources

Une fois le conteneur de la machine virtuelle créé sous Proxmox, nous avons procédé à l'allocation précise des ressources matérielles (CPU, RAM, Réseau) afin de garantir la stabilité et la fluidité du service hébergé.

1. Configuration du Processeur (CPU)

- **Configuration choisie** : 6 Cœurs.
- **Justification technique** : Nous avons attribué 6 cœurs virtuels à la machine pour lui permettre de gérer efficacement le *multitâche* et les charges de travail importantes.
 - Cela permet d'éviter les goulots d'étranglement (bottlenecks) lors des pics d'activité du service.

2. Configuration de la Mémoire Vive (RAM)

- **Configuration choisie** : 5 Go.
- **Justification technique** : La quantité de 5 Go a été définie pour offrir un confort d'exécution au système d'exploitation et aux applications, tout en conservant une marge de sécurité.
 - Cela évite le recours au "Swap" (mémoire virtuelle sur disque), qui ralentirait considérablement les performances.
 - Cette quantité est adaptée pour [Insérer ici le nom de l'OS ou du logiciel, ex: Windows Server / une base de données], qui nécessite une empreinte mémoire significative.

3. Configuration Réseau (Network)

- **Mode choisi** : Pont Linux.
- **Justification technique** : Nous avons opté pour le mode **Pont** plutôt que le NAT.
 - **Visibilité** : Ce mode place la VM directement sur le même segment réseau que l'hôte Proxmox et les autres équipements du réseau local.
 - **Adressage** : La VM peut ainsi obtenir sa propre adresse IP via le serveur DHCP du réseau ou être configurée avec une IP statique distincte. Cela facilite l'accès aux services hébergés depuis n'importe quel poste du réseau, sans configuration complexe de redirection de ports.

The screenshot displays the Proxmox Virtual Environment 9.0.3 web interface. The left sidebar shows a tree view of the datacenter contents, including various VMs and storage. The main panel shows the 'proxmox' node summary, which includes system information, resource usage, and a list of tasks.

Node 'proxmox' Summary:

- Uptime: 00:57:40
- CPU usage: 0.23% of 6 CPU(s)
- Load average: 0.00, 0.02, 0.00
- IO delay: 0.00%
- RAM usage: 32.62% (1.51 GiB of 4.64 GiB)
- KSM sharing: 0 B
- HD space: 84.43% (23.31 GiB of 27.60 GiB)
- SWAP usage: 0.00% (0 B of 4.64 GiB)
- CPU(s): 6 x 12th Gen Intel(R) Core(TM) i7-12650H (3 Sockets)
- Kernel Version: Linux 6.14.8-2-pve (2025-07-22T10:04Z)
- Boot Mode: Legacy BIOS
- Manager Version: pve-manager/9.0.3/025864202ebb6109
- Repository Status: No Proxmox VE repository enabled

Cluster log:

Start Time	End Time	Node	User name	Description	Status
Dec 07 17:40:25	Dec 07 17:40:25	proxmox	root@pam	Bulk start VMs and Containers	OK
Dec 06 18:08:56	Dec 06 18:10:55	proxmox	root@pam	Bulk shutdown VMs and Containers	OK
Dec 06 18:08:56	Dec 06 18:10:55	proxmox	root@pam	VM 114 - Shutdown	OK
Dec 06 18:08:56	Dec 06 18:09:05	proxmox	root@pam	VM 115 - Shutdown	OK
Dec 06 18:08:24	Dec 06 18:08:27	proxmox	root@pam	VM 115 - Start	OK

Phase 3 : Création des Machines Virtuelles Modèles (20%)

Définition des besoins en logiciels et environnement pour les TP

Pour les machines Linux :

Catégorie	Outils et Services intégrés
Services de base	ssh, nano, curl, wget
Services réseau	wireshark, bind9, dhcpd, tftpd, nginx, iptables
Services développement	git, python3, vscode
Services virtualisation	docker, virtualbox
Services gestion à distance	zabbix, nagios
Services télécoms	scapy
Services base de données	mariadb, phpmyadmin
Services sécurité / cyber	nmap, fail2ban, wireshark

Pour les machines Windows :

notepad ++

Création et configuration des machines virtuelles modèles

Afin de garantir l'homogénéité du parc informatique et la reproductibilité des déploiements, la procédure de création des machines virtuelles modèles a été entièrement **standardisée**.

L'intégralité du processus opérationnel — depuis l'installation du système d'exploitation via l'ISO jusqu'à la conversion technique en *Template* Proxmox — est documentée dans le fichier **README.md** joint au projet. Ce document fait office de référence unique pour assurer que chaque modèle généré respecte strictement les normes de configuration définies.

Phase 4 : Interface Web pour l'Accès à Distance (25%)

Déployer une solution utilisateur pour accéder aux VM de n'importe où

Pour répondre au besoin d'accessibilité sans contrainte géographique, nous avons développé et déployé une solution web sur mesure ("Flask-App") plutôt que d'utiliser une solution clé en main lourde. Cette application agit comme un portail unique pour les étudiants.

- **Architecture et Déploiement** : L'application repose sur une architecture conteneurisée orchestrée par **Docker Compose** pour faciliter son déploiement sur n'importe quel serveur. Elle est composée de trois services interconnectés : le conteneur applicatif (Python/Flask), une base de données MariaDB pour la persistance, et PhpMyAdmin pour la maintenance.
- **Technologie d'Accès Distant (NoVNC)** : Pour permettre l'accès "bureau à distance" directement dans le navigateur (sans installer de client lourd), nous avons intégré la technologie **NoVNC**. Le backend gère la complexité de la connexion via un mécanisme de proxy :
 1. L'application génère un ticket d'authentification unique via l'API Proxmox lorsqu'un étudiant demande l'accès.
 2. Ce ticket est injecté de manière transparente dans le navigateur de l'utilisateur via un cookie sécurisé (**PVEAuthCookie**).
 3. L'étudiant est redirigé vers la console virtuelle, lui donnant un contrôle total sur sa machine (clavier/souris/écran) depuis n'importe où.
- **Fonctionnalités Utilisateur** : L'interface offre un tableau de bord ergonomique (avec gestion du thème sombre/clair) permettant aux étudiants de gérer le cycle de vie de leurs machines : démarrage, arrêt, suppression et clonage rapide de templates pédagogiques via des "Linked Clones" pour économiser l'espace disque.

Mise en oeuvre de l'authentification et de la gestion des droits

La sécurisation de l'accès et la séparation des environnements étudiants étaient des enjeux critiques. Nous avons implémenté une gestion stricte des identités et des permissions.

- **Stratégie d'Authentification Hybride :**
 - **RADIUS :** Pour s'intégrer à l'annuaire de l'établissement, l'authentification principale repose sur le protocole RADIUS. Cela permet aux étudiants d'utiliser leurs identifiants habituels.
 - **Persistance Locale :** Lors de la première connexion réussie, l'utilisateur est automatiquement enregistré dans la base de données locale (**users**), assurant le lien entre son identité réseau et ses ressources virtuelles.
- **Gestion des Droits et des Rôles :** Le système distingue deux niveaux de privilèges gérés par le backend :
 - **Rôle "User" (Étudiant) :** Un cloisonnement strict est appliqué. Un étudiant ne peut voir et interagir qu'avec les machines qui lui appartiennent (vérification via **owner_id** dans la base de données) ou les templates publics.
 - **Rôle "Admin" :** Ce rôle dispose d'une vue globale sur l'ensemble du parc de VM et possède des droits étendus, incluant la gestion des utilisateurs Active Directory via des scripts PowerShell exécutés à distance (via SSH/Paramiko).
- **Sécurisation des Données Sensibles :** Pour garantir la sécurité de la mise en œuvre, aucune information critique (mots de passe de service, clés API) n'est stockée en clair. Nous utilisons la librairie de cryptographie **Fernet** pour chiffrer les secrets dans le code. Une clé de déchiffrement unique est générée et stockée dans un fichier **.env** isolé, permettant à l'application de s'authentifier de manière sécurisée auprès de l'infrastructure Proxmox et de la base de données.

Phase 5 : Documentation et Formation (10%)

Afin de garantir la pérennité du projet et l'autonomie des futurs administrateurs, l'intégralité des procédures d'installation, de déploiement et de maintenance a été formalisée dans une documentation technique exhaustive, consultable en annexe.

Phase 6 : Tests et Validation (5%)

Tests de performance et de sécurité

Une série de tests complets a été réalisée pour valider le bon fonctionnement du portail web et son interaction avec l'hyperviseur Proxmox. L'infrastructure est aujourd'hui pleinement opérationnelle et permet de gérer le cycle de vie complet des machines virtuelles.

- **Gestion multi-utilisateurs et Cloisonnement** : Les tests ont confirmé la stricte étanchéité des données. Un utilisateur connecté ne voit et n'agit que sur ses propres machines. L'administrateur conserve une visibilité globale.
- **Authentification Hybride** : Le mécanisme de double authentification (RADIUS pour les étudiants, base locale pour les administrateurs) fonctionne sans latence, avec un repli (fallback) automatique validé en cas d'indisponibilité du serveur RADIUS.
- **Opérations sur les VMs** : Les commandes envoyées via l'API Proxmox (Démarrage, Arrêt, Clonage de Template, Suppression) sont exécutées instantanément. Le système de "Linked Clone" permet un déploiement d'une nouvelle VM en moins de 10 secondes.
- **Accès Console (NoVNC)** : La redirection vers la console via le mécanisme de tickets (PVEAuthCookie) est fonctionnelle, permettant aux étudiants d'accéder à l'écran de leurs VMs directement depuis le navigateur sans configuration réseau complexe côté client.

Validation fonctionnelles de l'infrastructure

- **Sécurité des Données** : L'audit du code a permis de valider que les informations sensibles (mots de passe API, clés secrètes) ne sont jamais exposées en clair. L'utilisation de la librairie `cryptography` (Fernet) assure le chiffrement des identifiants dans la base de données et les fichiers de configuration (.env).
- **Tenue à la charge** : Des tests de simultanéité ont été effectués. Le serveur supporte sans difficulté plusieurs utilisateurs connectés simultanément effectuant des actions de clonage et d'accès console. L'architecture conteneurisée (Docker) du serveur web assure une isolation des processus et une consommation mémoire stable (environ 500 Mo pour l'ensemble de la stack web).

Étude de faisabilité : Déploiement pour la filière Réseaux & Télécoms

Contexte : L'objectif est d'ouvrir cette infrastructure aux 100 étudiants de la filière.

- **Capacité cible** : 100 Étudiants.
- **Quota** : 10 VMs par étudiant.
- **Total théorique** : 1000 Machines Virtuelles potentielles.

1. Analyse des besoins (Dimensionnement)

Bien que le parc total soit de 1000 VMs, elles ne tourneront jamais toutes en même temps. Nous basons le calcul sur un scénario d'usage intensif (type examen ou TP) :

- **Taux de machines utilisées en simultanées estimé** : 30% (soit 300 VMs allumées simultanément).
- **Profil moyen d'une VM étudiante (Linux/Windows Server Core)** : 2 vCPU, 4 Go RAM.

Calcul des ressources nécessaires (pour 300 VMs actives) :

- **RAM (Le facteur limitant)** : $300 \text{ VMs} * 4 \text{ Go} = 1200 \text{ Go}$ de RAM. Avec le mécanisme de "Ballooning" de Proxmox (récupération de mémoire inutilisée), nous pouvons réduire ce besoin de 30%, soit environ **800 Go de RAM requise**.
- **Stockage (Disque)** : Grâce aux "Linked Clones", 1000 VMs ne prennent pas 1000 fois la taille du disque. Seules les modifications (diff) sont stockées.
 - Estimation : 10 Go de données par VM active.
 - Besoin : $1000 \text{ VMs} * 10 \text{ Go} = 10 \text{ To}$ de stockage rapide (NVMe/SSD).
- **CPU** : Besoin d'un grand nombre de cœurs physiques pour gérer les files d'attente. Un ratio de surallocation (overcommit) de 1:4 est acceptable pour des TP.

2. Architecture Matérielle Proposée (Cluster 3 Nœuds)

Pour assurer la performance et la tolérance aux pannes, nous proposons un cluster de 3 serveurs physiques identiques.

Composant	Spécifications par Serveur (x3)	Justification
Serveur	Type Dell PowerEdge R750 ou HP ProLiant DL380	Matériel fiable, double alimentation, rackable.
Processeur	2x Intel Xeon Gold (ou AMD EPYC) - 32 Coeurs / 64 Threads	Permet de gérer ~100 VMs actives par serveur.
Mémoire (RAM)	512 Go DDR4 ECC	Total cluster : 1.5 To. Suffisant pour absorber les pics de charge.
Disques (Stockage)	4x 3.84 To SSD NVMe (RAID 5 ou ZFS RAIDZ1)	Total utile par serveur : ~11 To. Vitesse d'écriture critique pour 1000 VMs.
Réseau	2x 10 Gbps (SFP+)	Indispensable pour l'accès console fluide de 100 étudiants.

3. Estimation Budgétaire

Cette estimation est basée sur des prix publics moyens pour du matériel neuf (tarifs éducation).

Poste de dépense	Coût Unitaire (Est.)	Quantité	Coût Total
Serveurs Physiques (Complets CPU/RAM/Disques)	12 000 €	3	36 000 €
Switch Réseau 10G (Cœur de réseau)	1 500 €	1	1 500 €
Onduleur (UPS) (Protection électrique)	2 000 €	1	2 000 €
Licence Proxmox (Support Community - Optionnel)	110 € / an	3	330 € / an
TOTAL INVESTISSEMENT			~ 39 830 €

Conclusion de l'étude :

Pour un budget d'environ 40 000 €, la filière R&T pourrait disposer d'un Cloud Privé performant, autonome, capable de supporter les projets de virtualisation, de cybersécurité et d'administration système de ses 100 étudiants pour les 5 prochaines années. Le coût revient à environ 80 € par étudiant et par an (sur 5 ans), ce qui est très compétitif par rapport aux fournisseurs Cloud publics (AWS/Azure).

Phase 7 : Présentation et Livraison (5%)

Cette partie sera traitée directement le jour du passage

Ressources Transversales :

Réflexion sur les aspects de développement durables et de transition énergétique

Dans le cadre du projet de mise en place d'une infrastructure de salles de TP virtuelles, il est essentiel d'intégrer une réflexion sur le développement durable et la transition énergétique. Les choix technologiques et organisationnels réalisés influencent directement l'empreinte écologique et la consommation énergétique de l'infrastructure.

1. Optimisation des ressources grâce à la virtualisation

Le choix de Proxmox comme hyperviseur permet de mutualiser les ressources matérielles sur un nombre réduit de serveurs. Au lieu de multiplier les postes physiques pour chaque étudiant, la virtualisation concentre la charge de travail et optimise l'usage des CPU, de la mémoire et du stockage. Cette centralisation réduit la consommation énergétique globale et favorise une meilleure gestion des cycles de vie des machines virtuelles.

2. Réduction du matériel grâce à l'accès distant

La mise en place d'accès distant comme Interface Web permet aux étudiants d'accéder à distance aux environnements de TP depuis leur propre équipement (PC personnels, tablettes, etc.). Cela évite de sur équiper les salles informatiques avec des postes fixes énergivores, réduisant ainsi les coûts d'achat, d'entretien et la consommation électrique des postes traditionnels.

3. Gestion énergétique et supervision

Le projet intègre également des procédures de supervision qui permettent de surveiller l'utilisation des ressources et d'adapter la consommation en fonction des besoins réels. Les machines virtuelles inutilisées peuvent être mises en pause ou arrêtées automatiquement, limitant le gaspillage énergétique.

4. Pérennité et réduction des déchets électroniques

L'utilisation d'une infrastructure centralisée et virtualisée contribue à prolonger la durée de vie des équipements existants. Les étudiants peuvent accéder aux ressources même avec du matériel plus ancien, ce qui limite la production de déchets électroniques et réduit le besoin de renouveler fréquemment le parc informatique.

5. Contribution à la transition énergétique

En adoptant une infrastructure mutualisée et accessible à distance, l'établissement diminue sa consommation énergétique globale et encourage des pratiques numériques responsables. Couplée à une alimentation électrique issue de sources renouvelables (si le campus en dispose), cette approche s'inscrit pleinement dans les objectifs de transition énergétique.

6. Conclusion

Le projet de salles de TP virtuelles ne se limite pas à une innovation pédagogique et technique. Il constitue également une démarche durable en réduisant la consommation énergétique, en

optimisant l'usage des ressources matérielles en favorisant une gestion responsable du cycle de vie des équipements. Ces choix s'inscrivent dans une logique de transition énergétique et de responsabilité environnementale.

Synthèse en anglais

Executive Summary: Virtual Practical Work Rooms Infrastructure

1. Project Overview and Objectives

The main objective of this project was to design and deploy a complete infrastructure for virtual practical work (TP) rooms. The goals included installing a hypervisor, designing standardized virtual machine (VM) templates, and deploying a secure web interface to allow students to access their work environments remotely .

2. Hypervisor Selection and Infrastructure

After comparing VMware, Hyper-V, and KVM, the team selected **Proxmox VE** as the hypervisor.

Why Proxmox: It was chosen for being open-source, free, and offering a complete web interface with integrated snapshot and backup management, making it ideal for an academic context with budget constraints.

Configuration: The infrastructure was set up using nested virtualization (Proxmox inside VMware). The resources were allocated to ensure stability, with 6 vCores and 5 GB of RAM per instance, using Linux Bridge networking for direct LAN visibility.

3. Web Interface and User Access

Instead of a heavy off-the-shelf solution, a custom **"Flask-App"** was developed to manage user access.

Architecture: The solution is containerized using Docker (Python/Flask backend, MariaDB, and PhpMyAdmin).

Remote Access: It utilizes **NoVNC** technology to provide "clientless" remote desktop access directly via the browser using secure authentication tickets (PVEAuthCookie).

Features: Students can start, stop, and clone VMs using "Linked Clones" to save storage space.

4. Security and Authentication

Security was a critical focus of the project.

Hybrid Authentication: The system uses RADIUS to integrate with the school's directory for student login, while administrators are managed via a local database.

Data Protection: Strict isolation ensures students can only see their own machines. Critical data (API keys, secrets) is never stored in plain text but encrypted using the Fernet cryptography library.

5. Scalability Proposal (R&T Department)

A feasibility study was conducted to expand this project to the 100 students of the Network & Telecom department.

Capacity: The target is to support up to 1,000 VMs (10 per student).

Hardware: The proposal includes a 3-node cluster of Dell or HP servers with 1.5 TB of total RAM and 10Gbps networking.

Budget: The estimated investment is approximately **€39,830**, representing a cost of roughly €80 per student per year over five years, offering a competitive private cloud alternative.

6. Sustainability and Green IT

The project incorporates sustainable development principles. By using virtualization and resource mutualization, the infrastructure reduces energy consumption and hardware waste. The remote web access capability extends the lifespan of older student hardware, reducing electronic waste.